

LABORATORIO 06

Publicar solo si cuadra

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

En este laboratorio vas a cargar un lote de documentos en un costado de la tabla, revisarlo con controles de calidad y publicarlo solo si cuadra. Para eso Iceberg tiene ramas, que son libretas paralelas, y etiquetas, que son nombres para una página.

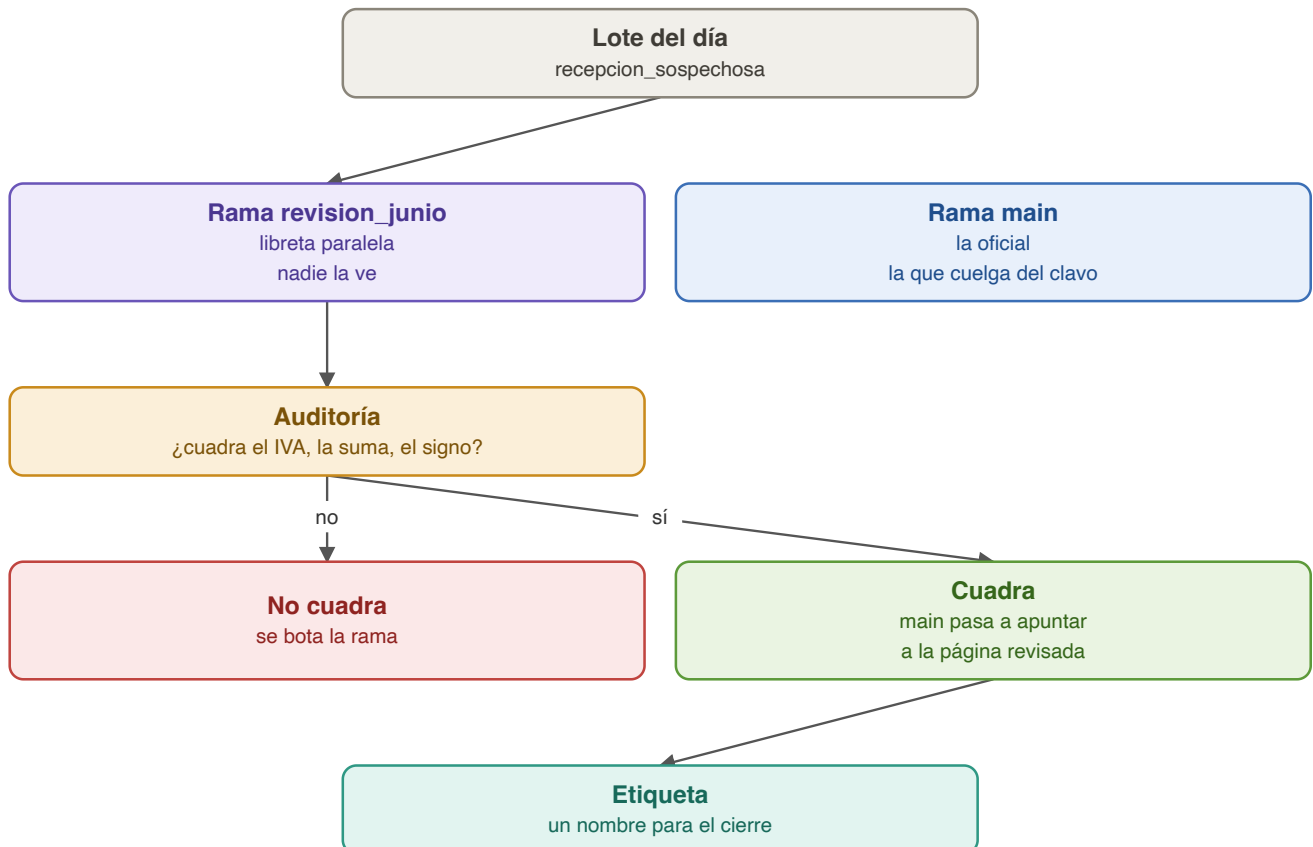
2 · El problema, en el almacén

El almacenero recibe cada día un lote de facturas y las anota en la libreta que cuelga del mostrador. Todos los que pasan la leen. Si un día el lote viene con errores, las cifras malas quedan a la vista antes de que alguien las revise, y cuando se descubre el error ya hay quien sacó cuentas con ellas.

3 · El problema en términos técnicos

Cuando un lote entra directo a la tabla, queda visible para todos los que la consultan en el mismo instante en que termina el `INSERT`. No hay un lugar intermedio donde revisarlo. Si el lote trae documentos con el IVA mal calculado o con el total mal sumado, esas cifras salen en los reportes hasta que alguien las detecta y las saca. Corregir después cuesta trabajo y cuesta credibilidad.

4 · El diagrama



Lote del día (gris). Son los documentos que llegan. Todavía nadie sabe si vienen bien.

Rama revision_junio (morado). Es una libreta paralela que nace igual a la oficial. El lote se carga aquí, y quien consulta la tabla no lo ve.

Rama main (azul). Es la libreta oficial, la que cuelga del clavo y la que leen los analistas. Mientras se revisa, no cambia.

Auditoría (amarillo). Sobre la rama se corren los controles. El IVA tiene que ser el 19 por ciento del neto, el total tiene que ser neto más exento más IVA, y solo las notas de crédito pueden tener monto negativo.

No cuadra (rojo). Si el lote falla, se bota la rama entera. La libreta oficial nunca vio esos datos.

Cuadra (verde). Si el lote pasa, la rama oficial pasa a apuntar a la página revisada. Eso es publicar.

Etiqueta (verde agua). Al final se le pone nombre a la página publicada, para volver a ella dentro de meses sin acordarse de ningún número.

5 · La solución

En el almacén, el lote del día se anota primero en una libreta aparte, que nadie ve. El almacenero la revisa con calma. Si algo no cuadra, bota esa libreta y la oficial queda como estaba. Si todo cuadra, pasa lo revisado a la oficial de un solo golpe. Y al cierre del mes le pega una etiqueta a esa página, para encontrarla después.

En Iceberg la libreta aparte es una **rama**. Se crea con `CREATE BRANCH`, se escribe nombrando la tabla con `branch_` y el nombre de la rama, se lee con `VERSION AS OF` y el nombre, se bota con `DROP BRANCH` y se publica con `REPLACE BRANCH main`. La etiqueta se crea con `CREATE TAG`. Ninguna de estas operaciones copia datos. Todas mueven punteros entre páginas que ya existen.

6 · Los pasos

- **Paso 0.** Creas la tabla `repcion_lab06` con un lote limpio de 1000 documentos.
- **Paso 1.** Cargas un lote sospechoso directo a la tabla, lo auditas, ves que no cuadra y lo deshaces volviendo a la primera página.
- **Paso 2.** Creas la rama `revision_junio`.
- **Paso 3.** Cargas el mismo lote en la rama y compruebas que la tabla oficial no cambió.
- **Paso 4.** Auditas la rama.
- **Paso 5.** Primero descartas la rama. Después creas otra, cargas solo los documentos que cuadran y la publicas.
- **Paso 6.** Le pones una etiqueta al cierre y miras ramas y etiquetas.

Si ya corriste este laboratorio, puedes volver a partir desde cero con el comando de abajo, desde la carpeta del repositorio. La celda 0.2 también borra la tabla.

```
bash
bin/reiniciar-lab.sh 06
```

Paso 0 · Preparar la tabla

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio van a quedar las tablas que crees?

Por qué ahora. Todo lo que hagas en este laboratorio se escribe en tu espacio de trabajo, `mi_espacio`. Hay que ponerse ahí antes de crear nada.

En el almacén. Es pararse frente a tu propio mostrador antes de abrir una libreta.

La sentencia, parte por parte.

- `%%sql` le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. El motor ignora todo lo que va después de dos guiones.
- `USE mi_espacio` deja a `mi_espacio` como espacio por omisión. Desde aquí, un nombre de tabla sin espacio adelante se busca en `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:23:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La última línea dice que la sentencia se ejecutó. Las tres de arriba son avisos de Spark al arrancar, y no son errores.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo avisos y errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar ese nivel. `sc` es el contexto de Spark. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop compilada para este sistema y usa la de Java. Funciona igual. `26/09/25 18:23:16` es la fecha y la hora del aviso, año 2026, mes 09, día 25, a las 18:23:16 UTC.

Listo. La sentencia se ejecutó. es el aviso que muestra el cuaderno cuando una sentencia no devuelve filas.

Celda 0.2 · Partir de cero

La pregunta. ¿Cómo se asegura que la tabla parta vacía aunque ya hayas corrido el laboratorio?

Por qué ahora. Si la tabla ya existe, la celda siguiente falla diciendo que el nombre está tomado.

En el almacén. Es sacar del mostrador la libreta vieja de este ejercicio, si quedó alguna.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no se queje si la tabla no existe.
- `repcion_lab06` es el nombre de la tabla, que se busca en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS repcion_lab06
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Si la tabla no existía, no pasó nada.

Celda 0.3 · La tabla con el lote limpio

La pregunta. ¿Cómo se crea la tabla ya cargada con el lote que está revisado y correcto?

Por qué ahora. Esta es la tabla oficial, la que consultan los analistas. Parte con un lote bueno.

En el almacén. Es abrir la libreta del mes y copiar en ella las facturas que ya se revisaron.

La sentencia, parte por parte.

- `CREATE TABLE repcion_lab06` crea la tabla.
- `USING iceberg` la hace tabla Iceberg, con páginas, ramas y etiquetas.
- `AS SELECT * FROM curso.repcion_limpia` la llena con todas las columnas y filas de `curso.repcion_limpia`, un lote limpio que trae el ambiente en el espacio `curso`.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE repcion_lab06 USING iceberg AS
SELECT * FROM curso.repcion_limpia
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y cargada.

Celda 0.4 · Cuánto hay

La pregunta. ¿Cuántos documentos tiene la tabla oficial y por cuánto suman?

Por qué ahora. Esta es la cifra que hoy sale en los reportes. Todo lo que viene se compara con ella.

En el almacén. Es contar las facturas de la libreta y sumar sus montos.

La sentencia, parte por parte.

- `count(*)` cuenta las filas, y `AS documentos` le pone nombre a esa columna.
- `sum(monto_total)` suma la columna `monto_total`, y `AS total_del_mes` le pone nombre.
- `FROM recepcion_lab06` es la tabla.

```
sql
%%sql
-- Celda 0.4
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_del_mes
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  total_del_mes
1000        9117709433.18
1 fila.
```

Cómo se lee. 1000 documentos que suman 9.117.709.433,18 pesos. Esa es la cifra oficial.

Paso 1 · El riesgo, mostrado

Celda 1.1 · Cargar el lote directo

La pregunta. ¿Qué pasa si el lote del día se carga directo a la tabla oficial?

Por qué ahora. Antes de aprender la forma segura hay que ver el riesgo en tu propia tabla.

En el almacén. Es anotar las facturas del día directo en la libreta del mostrador, sin revisarlas.

La sentencia, parte por parte.

- `INSERT INTO recepcion_lab06` agrega filas a la tabla oficial.
- `SELECT * FROM curso.recepcion_sospechosa` trae todas las filas del lote sospechoso, 500 documentos que trae el ambiente.

```
sql
%%sql
-- Celda 1.1
INSERT INTO recepcion_lab06 SELECT * FROM curso.recepcion_sospechosa
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

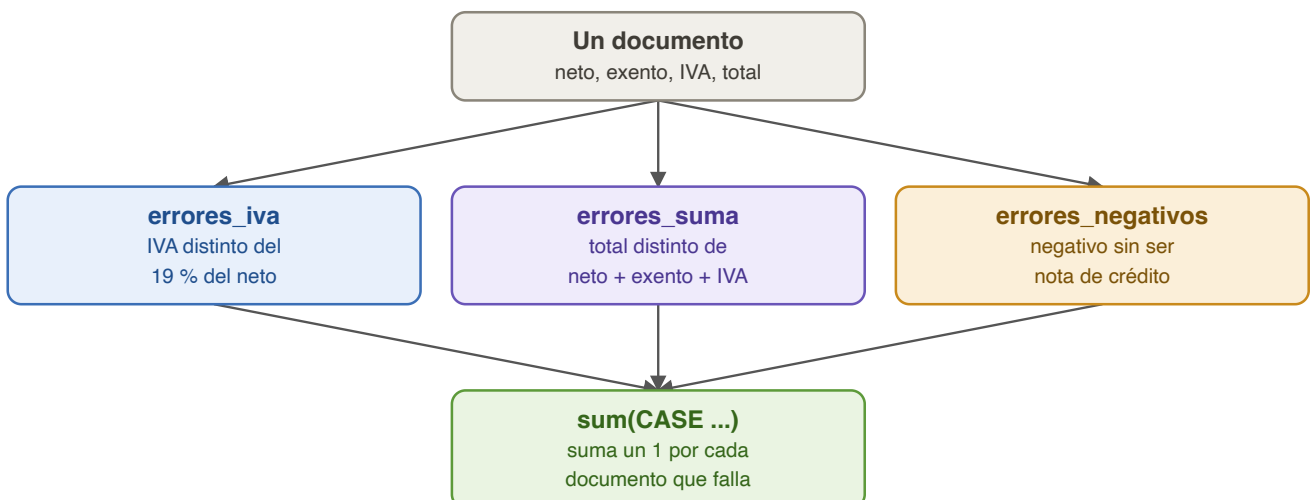
Cómo se lee. Entró, sin ningún aviso. Desde este momento cualquiera que consulte la tabla ve el lote nuevo.

Celda 1.2 · Los tres controles

La pregunta. ¿Cuántos documentos de la tabla rompen cada regla tributaria?

Por qué ahora. Ahora que el lote entró, se pasan los controles de calidad que debería haber pasado antes.

En el almacén. Es revisar factura por factura si el IVA, la suma y el signo están bien, y contar las que fallan.



Un documento (gris). Cada fila trae el monto neto, el exento, el IVA y el total.

errores_iva (azul). Cuenta los documentos cuyo IVA se aleja en más de dos pesos del 19 por ciento del neto.

errores_suma (morado). Cuenta los documentos cuyo total se aleja en más de dos pesos de neto más exento más IVA.

errores_negativos (amarillo). Cuenta los documentos con total negativo que no son nota de crédito. La nota de crédito, tipo 61, es el único documento que puede restar.

sum(CASE ...) (verde). Cada control le pone un 1 al documento que falla y un 0 al que cumple, y la suma de esos unos es la cantidad de documentos que rompen la regla.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta todas las filas.
- `CASE WHEN ... THEN 1 ELSE 0 END` es un si condicional. Si se cumple la condición vale 1, y si no, 0.
- `abs(monto_iva - round(monto_neto * 0.19)) > 2` calcula el 19 por ciento del neto, lo redondea con `round`, lo resta del IVA declarado y le quita el signo con `abs`. Si la diferencia pasa de 2 pesos, el IVA está mal. Los 2 pesos son una tolerancia por redondeo.
- `abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2` compara el total con la suma de sus partes, con la misma tolerancia.
- `monto_total < 0 AND tipo_dte <> 61` busca montos negativos en documentos que no son nota de crédito. `<>` significa distinto.
- `sum(...)` AS `errores_iva`, AS `errores_suma` y AS `errores_negativos` suman los unos de cada control y le ponen nombre a cada columna.
- `FROM recepcion_lab06` es la tabla oficial, que ahora incluye el lote nuevo.

```
sql
%%sql
-- Celda 1.2
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_neto * 0.19)) > 2
                 THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
                 THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                 THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1500        13         20           7
1 fila.
```

Cómo se lee. La tabla tiene 1500 documentos, los 1000 limpios más el lote. De ellos, 13 tienen el IVA mal, 20 tienen el total mal sumado y 7 tienen montos negativos que no deberían. Esos documentos ya están a la vista de todos.

Celda 1.3 · Cuántos documentos distintos fallan

La pregunta. ¿Cuántos documentos fallan al menos un control?

Por qué ahora. Un mismo documento puede fallar dos controles a la vez, así que las tres cifras no se suman.

En el almacén. Es separar en un montón todas las facturas que tienen algún error, sin contar dos veces la que tiene dos.

La sentencia, parte por parte.

- `count(*) AS documentos_que_no_cuadran` cuenta las filas que pasan el filtro.
- `WHERE` filtra las filas.
- Las tres condiciones son las mismas de la celda anterior, unidas con `OR`. Basta con que una se cumpla para que el documento cuente.

```
sql
%%sql
-- Celda 1.3
SELECT count(*) AS documentos_que_no_cuadran
FROM recepcion_lab06
WHERE abs(monto_iva - round(monto_neto * 0.19)) > 2
      OR abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
      OR (monto_total < 0 AND tipo_dte <> 61)
```

Lo que sale en pantalla.

```
text
documentos_que_no_cuadran
20
1 fila.
```

Cómo se lee. 20 documentos no cuadran, y están publicados. Varios fallan más de un control, por eso la cifra es menor que 13 más 20 más 7.

Celda 1.4 · Las páginas

La pregunta. ¿Qué páginas tiene la libreta después de la carga?

Por qué ahora. Para deshacer la carga hay que saber a qué página volver.

En el almacén. Es mirar la lista de páginas fechadas de la libreta.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué operación la creó.
- `FROM mi_espacio.recepcion_lab06.snapshots` lee la vista de sistema `.snapshots`. Se nombra con tres partes, el espacio, la tabla y la vista.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 1.4
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.recepcion_lab06.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
2338273078586975541  2026-09-25 18:23:25.633  append
7031022045785889757  2026-09-25 18:23:27.479  append
2 filas.
```

Cómo se lee. Dos páginas, las dos `append`, que significa que se agregaron filas. La primera, `2338273078586975541`, es la carga limpia, de las 18:23:25 UTC, las 15:23 en Chile. La segunda, `7031022045785889757`, es la carga del lote sospechoso.

Celda 1.5 · Volver a la página limpia

La pregunta. ¿Cómo se deshace la carga del lote sospechoso?

Por qué ahora. Los datos malos están publicados. Hay que sacarlos ya.

En el almacén. Es declarar que la página vigente vuelve a ser la de antes del lote.

La sentencia, parte por parte.

- `CALL` ejecuta un procedimiento del sistema.
- `spark_catalog.system.rollback_to_snapshot` es el procedimiento de Iceberg que vuelve la tabla a una página anterior. `spark_catalog` es el catálogo y `system` el espacio donde viven los procedimientos.
- `'mi_espacio.recepcion_lab06'` es la tabla, entre comillas y con su espacio adelante.
- `<TU_SNAPSHOT_ID>` es un marcador. En su lugar escribes, sin comillas, el `snapshot_id` de la primera fila de la celda 1.4. En la solución fue `2338273078586975541`. El tuyo va a ser otro.

```
sql
%%sql
-- Celda 1.5
CALL spark_catalog.system.rollback_to_snapshot('mi_espacio.recepcion_lab06', <TU_SNAPSHOT_ID>)
```

Lo que sale en pantalla.

```
text
previous_snapshot_id  current_snapshot_id
7031022045785889757  2338273078586975541
1 fila.
```

Cómo se lee. `previous_snapshot_id` es la página que mandaba antes, `7031022045785889757`, la del lote. `current_snapshot_id` es la que manda ahora, `2338273078586975541`, la carga limpia.

Celda 1.6 · La tabla quedó limpia

La pregunta. ¿La tabla volvió a estar bien?

Por qué ahora. Hay que comprobarlo con los mismos controles, no suponerlo.

En el almacén. Es revisar de nuevo la libreta después de volver a la página buena.

La sentencia, parte por parte. Es el mismo control de la celda 1.2.

- `count(*) AS documentos` cuenta las filas.
- Cada `sum(CASE WHEN ... THEN 1 ELSE 0 END)` cuenta los documentos que rompen una regla, el IVA, la suma o el signo, con la tolerancia de 2 pesos.
- `FROM recepcion_lab06` es la tabla oficial.

```
sql
%%sql
-- Celda 1.6
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_neto * 0.19)) > 2
                    THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
                    THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                    THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1000        0          0            0
1 fila.
```

Cómo se lee. 1000 documentos y cero errores en los tres controles. La tabla está limpia, pero los datos malos estuvieron publicados un rato, y en ese rato alguien pudo sacar un reporte.

Paso 2 · Crear la rama

Celda 2.1 · La rama de revisión

La pregunta. ¿Cómo se abre una libreta paralela donde cargar sin que nadie lo vea?

Por qué ahora. Ahora se hace bien. El lote se va a cargar en un costado.

En el almacén. Es abrir una libreta aparte, que empieza siendo una copia exacta de la oficial.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `CREATE BRANCH revision_junio` crea una rama llamada `revision_junio`, que nace apuntando a la misma página que la oficial.

```
sql
%%sql
-- Celda 2.1
ALTER TABLE recepcion_lab06 CREATE BRANCH revision_junio
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La rama quedó creada. No se copió ningún dato, solo se creó un puntero nuevo.

Celda 2.2 · Ramas y etiquetas de la tabla

La pregunta. ¿Qué ramas tiene ahora la tabla y a qué página apunta cada una?

Por qué ahora. Para ver que la rama nueva nace igual a la oficial.

En el almacén. Es mirar qué libretas cuelgan y en qué página está abierta cada una.

La sentencia, parte por parte.

- `SELECT name, type, snapshot_id` pide el nombre de cada referencia, si es rama o etiqueta, y la página a la que apunta.
- `FROM mi_espacio.recepcion_lab06.refs` lee la vista de sistema `.refs`, que lista ramas y etiquetas.
- `ORDER BY type, name` ordena por tipo y después por nombre.

```
sql
%%sql
-- Celda 2.2
SELECT name, type, snapshot_id
FROM mi_espacio.recepcion_lab06.refs
ORDER BY type, name
```

Lo que sale en pantalla.

```
text
name      type      snapshot_id
main      BRANCH    2338273078586975541
revision_junio  BRANCH    2338273078586975541
2 filas.
```

Cómo se lee. Dos ramas, `main` y `revision_junio`, las dos `BRANCH` y las dos en la página `2338273078586975541`. `main` es la oficial, la que ve todo el que consulta la tabla sin pedir nada especial.

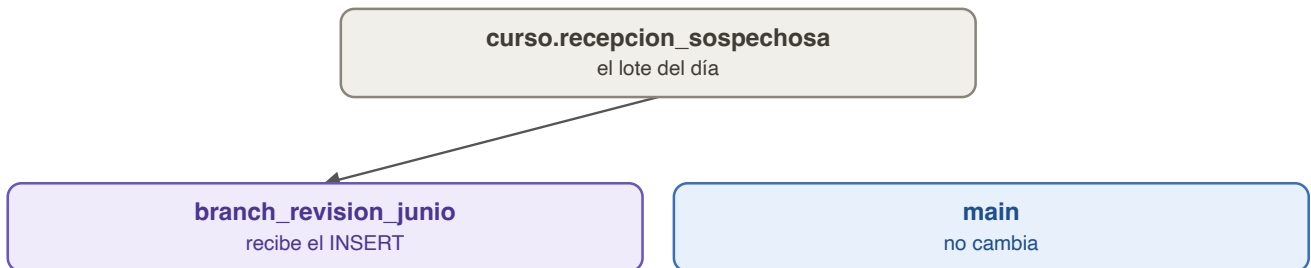
Paso 3 · Escribir en la rama

Celda 3.1 · El lote, en la rama

La pregunta. ¿Cómo se carga el lote en la rama y no en la tabla oficial?

Por qué ahora. Es el mismo lote sospechoso de antes, pero ahora va a un costado.

En el almacén. Es anotar las facturas del día en la libreta aparte.



curso.recepcion_sospechosa (gris). Es el mismo lote de la celda 1.1.

branch_revision_junio (morado). Es la forma de nombrar la rama como si fuera una tabla. El `INSERT` escribe ahí y la rama avanza a una página nueva.

main (azul). La rama oficial no recibe nada y sigue en la misma página.

La sentencia, parte por parte.

- `INSERT INTO mi_espacio.recepcion_lab06.branch_revision_junio` escribe en la rama. El nombre tiene tres partes, el espacio, la tabla y `branch_` seguido del nombre de la rama.
- `SELECT * FROM curso.recepcion_sospechosa` trae todas las filas del lote.

```
sql
%%sql
-- Celda 3.1
INSERT INTO mi_espacio.recepcion_lab06.branch_revision_junio
SELECT * FROM curso.recepcion_sospechosa
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El lote entró, pero en la rama.

Celda 3.2 · Leer la rama

La pregunta. ¿Qué hay en la rama?

Por qué ahora. Para comprobar que el lote quedó en la rama.

En el almacén. Es abrir la libreta aparte y contar.

La sentencia, parte por parte.

- `count(*) AS documentos` y `sum(monto_total) AS total_del_mes` cuentan y suman.
- `FROM recepcion_lab06 VERSION AS OF 'revision_junio'` lee la tabla como está en la rama. `VERSION AS OF` recibe el nombre de la rama entre comillas.

```
sql
%%sql
-- Celda 3.2
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_del_mes
FROM recepcion_lab06 VERSION AS OF 'revision_junio'
```

Lo que sale en pantalla.

```
text
documentos  total_del_mes
1500        13693496696.68
1 fila.
```

Cómo se lee. En la rama hay 1500 documentos que suman 13.693.496.696,68 pesos. Son los 1000 limpios más el lote.

Celda 3.3 · Leer la tabla oficial

La pregunta. ¿Qué ve quien consulta la tabla oficial?

Por qué ahora. Es la comprobación que importa. El lote está cargado, pero no debería verse.

En el almacén. Es abrir la libreta del mostrador y contar.

La sentencia, parte por parte.

- `count(*) AS documentos` y `sum(monto_total) AS total_del_mes` cuentan y suman.
- `FROM recepcion_lab06` lee la tabla sin pedir versión, así que lee `main`.

```
sql
%%sql
-- Celda 3.3
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_del_mes
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  total_del_mes
1000        9117709433.18
1 fila.
```

Cómo se lee. 1000 documentos y el mismo total de la celda 0.4. Para los analistas la tabla no ha cambiado.

Paso 4 · Auditar

Celda 4.1 · Los controles, en la rama

La pregunta. ¿Cuántos documentos de la rama rompen cada regla?

Por qué ahora. Ahora sí se audita antes de publicar.

En el almacén. Es revisar la libreta aparte con los mismos controles de siempre.

La sentencia, parte por parte. Es el control de la celda 1.2, con otra fuente.

- `count(*) AS documentos` cuenta las filas.
- Cada `sum(CASE WHEN ... THEN 1 ELSE 0 END)` cuenta los documentos que fallan una regla, el IVA con `abs` y `round`, la suma de las partes y el signo de los que no son tipo 61.
- `FROM recepcion_lab06 VERSION AS OF 'revision_junio'` lee la rama.

```
sql
%%sql
-- Celda 4.1
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_netto * 0.19)) > 2
                THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_netto + monto_exento + monto_iva)) > 2
                THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06 VERSION AS OF 'revision_junio'
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1500        13         20           7
1 fila.
```

Cómo se lee. Los mismos descuadres de la celda 1.2, 13, 20 y 7, pero esta vez están en la rama y nadie los vio.

Celda 4.2 · El veredicto

La pregunta. ¿Cuántos documentos de la rama fallan algún control?

Por qué ahora. Es la cifra que decide si el lote pasa.

En el almacén. Es contar las facturas malas de la libreta aparte.

La sentencia, parte por parte.

- `count(*) AS documentos_que_no_cuadran` cuenta.
- `FROM recepcion_lab06 VERSION AS OF 'revision_junio'` lee la rama.
- `WHERE` con las tres condiciones unidas por `OR` deja las filas que fallan al menos una.

```
sql
%%sql
-- Celda 4.2
SELECT count(*) AS documentos_que_no_cuadran
FROM recepcion_lab06 VERSION AS OF 'revision_junio'
WHERE abs(monto_iva - round(monto_neto * 0.19)) > 2
      OR abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
      OR (monto_total < 0 AND tipo_dte <> 61)
```

Lo que sale en pantalla.

```
text
documentos_que_no_cuadran
20
1 fila.
```

Cómo se lee. 20 documentos no cuadran. El lote no pasa.

Paso 5 · Decidir

Celda 5.1 · Descartar la rama

La pregunta. ¿Cómo se bota el lote entero sin tocar la tabla oficial?

Por qué ahora. Es el primero de los dos caminos, el más simple.

En el almacén. Es tirar la libreta aparte al canasto.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `DROP BRANCH revision_junio` borra la rama.

```
sql
%%sql
-- Celda 5.1
ALTER TABLE recepcion_lab06 DROP BRANCH revision_junio
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La rama ya no existe.

Celda 5.2 · La tabla oficial, intacta

La pregunta. ¿La tabla oficial quedó como estaba?

Por qué ahora. Hay que confirmar que botar la rama no tocó nada más.

En el almacén. Es revisar la libreta del mostrador después de tirar la otra.

La sentencia, parte por parte. Es el control de la celda 1.2 sobre la tabla oficial.

- `count(*) AS documentos` cuenta las filas.
- Cada `sum(CASE WHEN ... THEN 1 ELSE 0 END)` cuenta los que fallan una regla.
- `FROM recepcion_lab06` lee `main`.

```
sql
%%sql
-- Celda 5.2
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_neto * 0.19)) > 2
                THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
                THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1000        0          0          0
1 fila.
```

Cómo se lee. 1000 documentos y cero descuadres. Nadie vio nunca los datos malos.

Celda 5.3 · Qué ramas quedan

La pregunta. ¿Qué ramas tiene la tabla ahora?

Por qué ahora. Para ver que la rama se fue.

En el almacén. Es mirar qué libretas cuelgan.

La sentencia, parte por parte.

- `SELECT name, type` pide el nombre y el tipo.
- `FROM mi_espacio.recepcion_lab06.refs` es la vista de ramas y etiquetas.
- `ORDER BY type, name` ordena.

```
sql
%%sql
-- Celda 5.3
SELECT name, type
FROM mi_espacio.recepcion_lab06.refs
ORDER BY type, name
```

Lo que sale en pantalla.

```
text
name  type
main  BRANCH
1 fila.
```

Cómo se lee. Solo queda `main`.

Celda 5.4 · Una rama nueva

La pregunta. ¿Cómo se abre otra libreta aparte para el segundo camino?

Por qué ahora. Descartar 1500 documentos por 20 malos no siempre conviene. El segundo camino es sanear y publicar.

En el almacén. Es abrir otra libreta aparte, limpia.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `CREATE BRANCH revision_junio_v2` crea una rama nueva, con otro nombre, que nace en la misma página que `main`.

```
sql
%%sql
-- Celda 5.4
ALTER TABLE recepcion_lab06 CREATE BRANCH revision_junio_v2
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La rama quedó creada.

Celda 5.5 · Cargar solo lo que cuadra

La pregunta. ¿Cómo se carga el lote dejando fuera los documentos malos?

Por qué ahora. Es el saneamiento. Entra lo bueno, y lo malo se queda afuera.

En el almacén. Es anotar en la libreta aparte solo las facturas que pasan la revisión.

La sentencia, parte por parte.

- `INSERT INTO mi_espacio.recepcion_lab06.branch_revision_junio_v2` escribe en la rama nueva.
- `SELECT * FROM curso.recepcion_sospechosa` trae el lote.
- `WHERE` deja pasar solo los buenos. Es el control al revés. El IVA tiene que estar a 2 pesos o menos del 19 por ciento del neto, con `<= 2`, y lo mismo el total.
- `AND` exige que se cumplan todas las condiciones.
- `(monto_total >= 0 OR tipo_dte = 61)` deja pasar los montos positivos o cero, y los negativos solo si son nota de crédito.

```
sql
%%sql
-- Celda 5.5
INSERT INTO mi_espacio.recepcion_lab06.branch_revision_junio_v2
SELECT * FROM curso.recepcion_sospechosa
WHERE abs(monto_iva - round(monto_neto * 0.19)) <= 2
      AND abs(monto_total - (monto_neto + monto_exento + monto_iva)) <= 2
      AND (monto_total >= 0 OR tipo_dte = 61)
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El lote saneado quedó en la rama.

Celda 5.6 · Auditar la rama nueva

La pregunta. ¿La rama saneada pasa los controles?

Por qué ahora. No se publica nada sin auditarlo.

En el almacén. Es revisar la libreta aparte antes de pasarla a la oficial.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta las filas.
- Cada `sum(CASE WHEN ... THEN 1 ELSE 0 END)` cuenta los que fallan una regla.
- `FROM recepcion_lab06 VERSION AS OF 'revision_junio_v2'` lee la rama nueva.

```
sql
%%sql
-- Celda 5.6
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_neto * 0.19)) > 2
                    THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
                    THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                    THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06 VERSION AS OF 'revision_junio_v2'
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1480        0          0             0
1 fila.
```

Cómo se lee. 1480 documentos y cero descuadres. Son los 1000 limpios más los 480 buenos del lote. Este lote sí pasa.

Celda 5.7 · El número de la página a publicar

La pregunta. ¿En qué página está la rama revisada?

Por qué ahora. Para publicar hay que decirle a `main` a qué página apuntar, y eso se hace con el número de la página.

En el almacén. Es anotar el número de la página de la libreta aparte que se va a pasar a la oficial.

La sentencia, parte por parte.

- `SELECT snapshot_id` pide el número de página.
- `FROM mi_espacio.recepcion_lab06.refs` es la vista de ramas y etiquetas.
- `WHERE name = 'revision_junio_v2'` se queda con la rama revisada.

```
sql
%%sql
-- Celda 5.7
SELECT snapshot_id
FROM mi_espacio.recepcion_lab06.refs
WHERE name = 'revision_junio_v2'
```

Lo que sale en pantalla.

```
text
snapshot_id
4004546388929856598
1 fila.
```

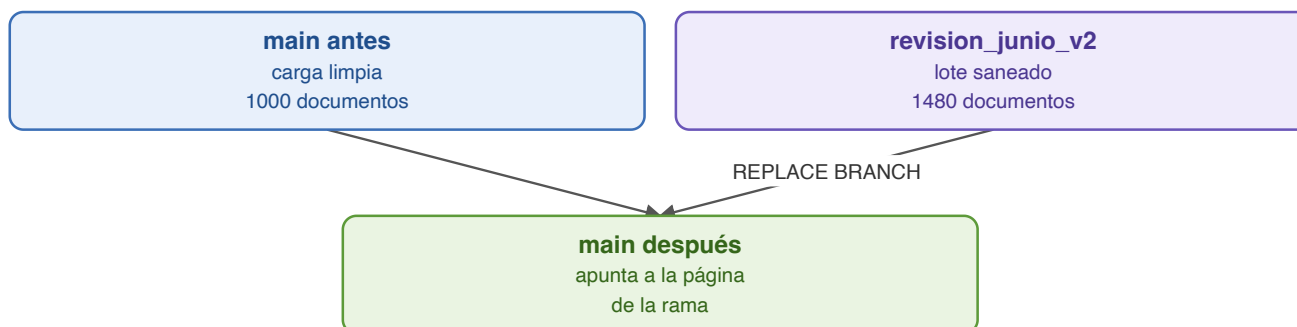
Cómo se lee. La rama está en la página `4004546388929856598`. En tu pantalla va a ser otro número.

Celda 5.8 · Publicar

La pregunta. ¿Cómo pasa lo revisado a la tabla oficial?

Por qué ahora. La rama cuadra. Es el momento de publicar.

En el almacén. Es declarar que la libreta oficial ahora está en la página revisada.



main antes (azul). La oficial está en la página de la carga limpia.

revision_junio_v2 (morado). La rama está en la página con el lote saneado.

main después (verde). `REPLACE BRANCH main` mueve la oficial a la página de la rama. No se copia ningún dato, solo cambia a qué página apunta.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `REPLACE BRANCH main` mueve la rama `main`.
- `AS OF VERSION` dice a qué página moverla.
- `<TU_SNAPSHOT_DE_LA_RAMAS>` es un marcador. En su lugar escribes, sin comillas, el número que sacaste en la celda 5.7. En la solución fue `4004546388929856598`.

```
sql
%%sql
-- Celda 5.8
ALTER TABLE recepcion_lab06 REPLACE BRANCH main AS OF VERSION <TU_SNAPSHOT_DE_LA_RAMAS>
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Publicado.

Celda 5.9 · La tabla oficial, publicada

La pregunta. ¿Qué ven ahora los analistas?

Por qué ahora. Hay que comprobar que lo publicado es lo revisado.

En el almacén. Es revisar la libreta del mostrador después de publicar.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta las filas.
- Cada `sum(CASE WHEN ... THEN 1 ELSE 0 END)` cuenta los que fallan una regla.
- `FROM recepcion_lab06` lee `main`.

```
sql
%%sql
-- Celda 5.9
SELECT count(*) AS documentos,
       sum(CASE WHEN abs(monto_iva - round(monto_neto * 0.19)) > 2
                    THEN 1 ELSE 0 END) AS errores_iva,
       sum(CASE WHEN abs(monto_total - (monto_neto + monto_exento + monto_iva)) > 2
                    THEN 1 ELSE 0 END) AS errores_suma,
       sum(CASE WHEN monto_total < 0 AND tipo_dte <> 61
                    THEN 1 ELSE 0 END) AS errores_negativos
FROM recepcion_lab06
```

Lo que sale en pantalla.

```
text
documentos  errores_iva  errores_suma  errores_negativos
1480        0          0              0
1 fila.
```

Cómo se lee. 1480 documentos y cero descuadres en la tabla oficial. En ningún momento estuvo publicado un dato malo.

Celda 5.10 · Botar la rama que ya cumplió

La pregunta. ¿Qué se hace con la rama después de publicar?

Por qué ahora. Lo que tenía ya está en `main`. Una rama vieja solo estorba.

En el almacén. Es guardar la libreta aparte una vez que lo suyo quedó en la oficial.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `DROP BRANCH revision_junio_v2` borra la rama. Las páginas siguen, porque `main` apunta a ellas.

```
sql
%%sql
-- Celda 5.10
ALTER TABLE recepcion_lab06 DROP BRANCH revision_junio_v2
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La rama se borró.

Paso 6 · Etiquetar el cierre

Celda 6.1 · Ponerle nombre al cierre

La pregunta. ¿Cómo se marca el estado con que se cierra el mes?

Por qué ahora. Dentro de meses alguien va a preguntar por el cierre de junio, y es más fácil pedirlo por nombre que por un número de página.

En el almacén. Es pegarle una etiqueta a la página del cierre que diga cierre de junio.

La sentencia, parte por parte.

- `ALTER TABLE recepcion_lab06` cambia algo de la tabla.
- `CREATE TAG cierre_junio_2026` crea una etiqueta con ese nombre sobre la página vigente.

```
sql
%%sql
-- Celda 6.1
ALTER TABLE recepcion_lab06 CREATE TAG cierre_junio_2026
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La etiqueta quedó creada.

Celda 6.2 · Pedir la tabla por la etiqueta

La pregunta. ¿Qué hay en la página del cierre?

Por qué ahora. Para ver que la etiqueta se usa igual que una rama.

En el almacén. Es abrir la libreta en la página que tiene la etiqueta.

La sentencia, parte por parte.

- `count(*) AS documentos` y `sum(monto_total) AS total_del_mes` cuentan y suman.
- `FROM recepcion_lab06 VERSION AS OF 'cierre_junio_2026'` lee la tabla en la página de la etiqueta.

```
sql
%%sql
-- Celda 6.2
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_del_mes
FROM recepcion_lab06 VERSION AS OF 'cierre_junio_2026'
```

Lo que sale en pantalla.

```
text
documentos  total_del_mes
1480        13576071743.67
1 fila.
```

Cómo se lee. 1480 documentos que suman 13.576.071.743,67 pesos. Ese es el cierre de junio, y va a seguir siéndolo aunque la tabla cambie después.

Celda 6.3 · Ramas y etiquetas al final

La pregunta. ¿Qué referencias tiene la tabla al terminar?

Por qué ahora. Es la última mirada.

En el almacén. Es mirar qué libretas cuelgan y qué etiquetas tiene la oficial.

La sentencia, parte por parte.

- `SELECT name, type, snapshot_id` pide nombre, tipo y página.
- `FROM mi_espacio.recepcion_lab06.refs` es la vista de ramas y etiquetas.
- `ORDER BY type, name` ordena por tipo y nombre.

```
sql
%%sql
-- Celda 6.3
SELECT name, type, snapshot_id
FROM mi_espacio.recepcion_lab06.refs
ORDER BY type, name
```

Lo que sale en pantalla.

```
text
name          type      snapshot_id
main          BRANCH   4004546388929856598
cierre_junio_2026 TAG      4004546388929856598
2 filas.
```

Cómo se lee. Una rama, `main`, y una etiqueta, `cierre_junio_2026`, las dos en la página `4004546388929856598`. `BRANCH` es rama y `TAG` es etiqueta.

Preguntas frecuentes

¿Crear una rama copia los datos?

No. Una rama es un puntero a una página. Al crearla apunta a la misma página que `main`, y los papelitos son los mismos. Solo cuando escribes en la rama se agregan papelitos nuevos, y esos los ve solo la rama.

¿Qué diferencia hay entre una rama y una etiqueta?

Una rama avanza cada vez que escribes en ella. Una etiqueta queda fija en la página donde la creaste. La rama sirve para trabajar aparte, y la etiqueta para recordar un estado.

¿Por qué hay que copiar el número de la página para publicar?

Porque en esta versión de Iceberg `REPLACE BRANCH main AS OF VERSION` recibe un número de página y no el nombre de una rama. Por eso se saca el número de `.refs`. En versiones más nuevas existe `CALL system.fast_forward`, que publica una rama por su nombre, pero aquí no está disponible.

¿Qué pasa con la tabla oficial mientras se revisa la rama?

Nada. Quien la consulta sigue viendo `main`, y puede seguir cargándose en `main` al mismo tiempo. Las dos libretas avanzan por separado.

¿La etiqueta protege la página si alguien bota páginas viejas?

Sí. Una página con etiqueta no se bota con la limpieza de páginas viejas mientras la etiqueta exista. Por eso las etiquetas sirven para los cierres que hay que poder mostrar después.

¿Por qué la tolerancia de dos pesos?

Porque el IVA se redondea a pesos, y la suma de partes redondeadas puede diferir en uno o dos pesos del total. Sin tolerancia, documentos correctos aparecerían como malos.

¿Qué pasa si publico sin auditar?

Técnicamente funciona igual. Las ramas no obligan a revisar, solo dan el lugar donde hacerlo. La regla de revisar antes de publicar la pone el proceso de carga.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Entre cargar y publicar ahora hay un espacio donde revisar. Antes, cargar era publicar.

En el almacén. El lote del día se anota en una libreta aparte, se revisa, y solo si cuadra pasa a la oficial. Si no cuadra, se tira la libreta aparte y la oficial ni se entera. El cierre del mes queda con una etiqueta para encontrarlo después.

Los números de la solución ejecutada.

Momento	Documentos en la tabla oficial	Documentos que no cuadran
Carga limpia	1000	0
Lote cargado directo	1500	20
Después de volver a la primera página	1000	0
Lote en la rama, tabla oficial	1000	0
Después de publicar la rama saneada	1480	0

Lo que necesitas	La sentencia
Abrir una libreta paralela	<code>ALTER TABLE t CREATE BRANCH nombre</code>
Escribir en ella	<code>INSERT INTO espacio.tabla.branch_nombre SELECT ...</code>
Leerla	<code>SELECT ... FROM t VERSION AS OF 'nombre'</code>
Botarla	<code>ALTER TABLE t DROP BRANCH nombre</code>
Publicarla	<code>ALTER TABLE t REPLACE BRANCH main AS OF VERSION numero</code>
Marcar un estado	<code>ALTER TABLE t CREATE TAG nombre</code>
Ver ramas y etiquetas	<code>SELECT ... FROM espacio.tabla.refs</code>

Un lote con errores deja de ser un problema de credibilidad y pasa a ser un trámite. Se audita en la rama, se descarta o se sanea, y a la tabla oficial llega solo lo que cuadra.